

RESUMÃO PARA A PROVA – HTTP + WIRESHARK + NGINX para PHP + NGINX para HTTP/2

1. PROTOCOLO HTTP — Conceitos Principais

O que é HTTP

- HTTP = *HyperText Transfer Protocol*
- Protocolo usado para comunicação entre **cliente (browser)** e **servidor web**.
- Funciona **sobre TCP**, que é confiável e orientado à conexão.

Fluxo básico:

1. Cliente envia uma **requisição HTTP**.
2. Servidor responde com uma **resposta HTTP**.
3. Conexão é encerrada (em HTTP/1.0) ou permanece aberta (HTTP/1.1+).

Tipos de Conexões

♦ Conexão não persistente (HTTP/1.0)

- Abre uma conexão TCP para **cada** pedido/resposta.
- Lenta e gera mais overhead.
- Fórmula: $N \text{ objetos} \rightarrow N \text{ conexões} + 1$.

♦ Conexão persistente (HTTP/1.1 — padrão)

- Uma conexão TCP permanece aberta para múltiplas requisições.
 - Muito mais eficiente.
-

✓ Formato das Mensagens HTTP

📌 Mensagem de Pedido (Request)

Estrutura:

1. Linha de solicitação

- **Método** → GET, POST, PUT, DELETE...
- **URL** → recurso solicitado
- **Versão** → HTTP/1.0, HTTP/1.1

2. Cabeçalhos

- Informações extras (User-Agent, Accept, Host, etc.)

3. Linha em branco

4. Corpo (Body)

- Usado em POST/PUT.
-

📌 Mensagem de Resposta (Response)

Estrutura:

1. Linha de status

- Versão HTTP
- Código de status

- Texto explicativo
 - 2. **Cabeçalhos**
 - 3. **Linha em branco**
 - 4. **Corpo (conteúdo da página)**
-

Códigos de Status HTTP

1xx: Informativo

- 100 Continue

2xx: Sucesso

- 200 OK
- 201 Created

3xx: Redirecionamento

- 301 Moved Permanently
- 302 Found
- 304 Not Modified

4xx: Erro do Cliente

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

5xx: Erro do Servidor

- 500 Internal Server Error
 - 503 Service Unavailable
-

Versões do HTTP

HTTP/0.9

- Apenas método GET
- Texto puro

HTTP/1.0

- Não persistente
- Metadados simples

HTTP/1.1

- Conexões persistentes (padrão)
- Pipeline
- Mais cabeçalhos e melhor cache

HTTP/2

- Binário
 - Multiplexação (vários fluxos na mesma conexão)
 - Muito mais rápido
 - Frames ao invés de texto puro
-



2. WIRESHARK — Conceitos Principais



O que é Wireshark

- Ferramenta *sniffer* para capturar e analisar pacotes da rede.
 - Gratuito e open-source.
 - Utilizado para:
 - Diagnóstico de rede
 - Segurança
 - Análise de protocolos
 - Testes e estudos
-



Como usar o Wireshark

1. Escolher interface de rede

- Loopback → 127.0.0.1 (análise local)
- Rede física (Wi-Fi, Ethernet)

2. Iniciar captura

- Botão azul (“Start”)
- Ou menu Capture → Start

3. Aplicar filtros

Exemplos úteis:

- `http`

- `tcp.port == 80`
- `ip.addr == 127.0.0.1`

✓ Campos da Tela Principal

Cada linha representa **um pacote**:

Campo	Significado
No.	Número do pacote
Time	Tempo relativo
Source	IP de origem
Destination	IP de destino
Protocol	HTTP, TCP, IP...
Length	Tamanho do pacote
Info	Resumo da operação

✓ Análise de Protocolos (camadas TCP/IP)

IP

- Endereçamento
- Roteamento

TCP

- Estabelece conexão (3-way handshake)
- Controle de fluxo e confiabilidade

HTTP

- Conteúdo da aplicação
 - Requests e responses podem ser vistos no Wireshark
-

3-Way Handshake (TCP)

1. **SYN**
2. **SYN + ACK**
3. **ACK**

Encerramento: **FIN/ACK**

Analisando HTTP no Wireshark

- Verificar se o servidor está rodando (ex: NGINX)
 - Capturar tráfego enquanto acessa a página
 - Clicar em uma linha HTTP para ver:
 - Cabeçalhos
 - Métodos
 - Códigos de status
 - Corpo da resposta (HTML)
-

Relação HTTP → TCP → IP

- **HTTP**: conteúdo (requisição/resposta)
- **TCP**: transporte confiável

- **IP**: endereçamento e roteamento

Tudo isso aparece no Wireshark quando analisado.

3. Configuração do NGINX para PHP (Páginas Dinâmicas)

O objetivo é permitir que o servidor **NGINX** (servidor web) consiga processar e servir páginas dinâmicas escritas em **PHP**¹.

Conceitos Chave

Conceito	Descrição
NGINX e PHP	O NGINX não processa PHP nativamente; ele requer um programa externo para atuar como ponte entre o interpretador PHP e o servidor web ² .
php-fpm	PHP FastCGI Process Manager . É o software essencial instalado para processar solicitações PHP ³ . Ele funciona como um gerenciador de processos FastCGI (protocolo de comunicação entre o servidor web e o PHP) ⁴ .
php-mysql	Módulo do PHP necessário para permitir a comunicação com bancos de dados baseados em MySQL ⁵ .

Passos de Configuração (Comandos e Diretivas)

1. **Instalação do PHP e Módulo MySQL (Passo 1):**
 - Instalar: `sudo apt install php8.1-fpm php-mysql.`
 - Verificar status do PHP: `systemctl status php8.1-fpm.service.`

2. Edição do Server Block (/etc/nginx/sites-available/dominio) (Passo 2):

- **Diretivas de Configuração Básica:**

- `listen 80;`: Porta padrão para HTTP.
- `server_name`: Define o nome do domínio para o qual o bloco de servidor deve responder.
- `root`: Define o diretório raiz onde os arquivos do site estão armazenados.
- `index`: Define a ordem de prioridade dos arquivos de índice (ex: `index index.html index.php;`).
- `access_log` e `error_log`: Linhas adicionadas para armazenar logs de acesso e erros, essenciais para análise.

- **Bloco de Localização /:**

- `location / { try_files $uri $uri/ =404; }`: Verifica se o arquivo ou diretório solicitado existe. Se não for encontrado, retorna o erro **404 (Not Found)**.

- **Bloco de Localização para PHP:**

- `location ~ \.php$ { ... }`: Lida com o processamento real de PHP.
- Aponta para o PHP-FPM: `include snippets/fastcgi-php.conf;` e `fastcgi_pass unix:/var/run/php/php8.1-fpm.sock;`

- **Bloco de Segurança para .htaccess:**

- `location ~ /\.ht { deny all; }`: Impede que arquivos `.htaccess` (que o NGINX não processa) sejam apresentados aos visitantes.

3. Verificação e Restart:

- Verificar sintaxe do NGINX: `sudo nginx -t.`
- Reiniciar NGINX: `sudo systemctl restart nginx.`

4. Configuração do NGINX para HTTP/2 e HTTPS (Segurança)

A configuração para o protocolo **HTTP/2** é crucial e depende diretamente da habilitação do protocolo **HTTPS**.

Conceitos ChavePassos de Configuração

Conceito	Descrição
HTTP/2 e HTTPS	Navegadores modernos suportam HTTP/2 apenas sobre conexões seguras, ou seja, HTTPS.
HTTPS	É o protocolo HTTP rodando sobre a Camada de Sockets Segura (SSL) . Garante confidencialidade , autenticação (identidade do usuário) e integridade dos dados .
Certificado Digital SSL/TLS	É a identidade eletrônica do servidor. Sites que o utilizam podem estabelecer comunicação segura. É visível no navegador pelo ícone de cadeado . Seu objetivo é garantir que os dados sejam criptografados .
Chaves Criptográficas	O certificado utiliza um par de chaves: chave privada (para criptografar dados que atestam a identidade, conhecida apenas pelo autorizado) e chave pública (compartilhada para decodificar a criptografia).
Certificado Autoassinado	Certificado gerado localmente (e não por uma Autoridade Certificadora) para fins de teste. É gerado usando a ferramenta OpenSSL .

1. Geração do Certificado Autoassinado (Passo 2):

- Comando OpenSSL (Exemplo): `sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/nginx-selfsigned.key -out /etc/ssl/certs/nginx-selfsigned.crt`.
- `openssl`: Ferramenta básica para criar e gerenciar certificados.
- `-x509`: Cria um certificado autoassinado.
- `-newkey rsa:2048`: Gera uma nova chave RSA.
-

2. Criação de Arquivos de Configuração SSL (Snippets - Passos 3 e 4):

- `self-signed.conf`: Armazena os caminhos para o certificado (`ssl_certificate`) e a chave privada (`ssl_certificate_key`).
- `ssl-params.conf`: Contém configurações de segurança, como `ssl_protocols TLSv1.3`; e o caminho para o parâmetro Diffie-Hellman (`ssl_dhparam`).

3. Ajuste do Server Block (HTTPS) (Passo 6):

- O servidor deve escutar na **porta 443** (padrão para HTTPS) e incluir as configurações SSL/TLS:

4. Nginx

None

```
server {  
    listen 443 ssl;  
    listen [::]:443 ssl;  
    include /etc/nginx/snippets/self-signed.conf;  
    include /etc/nginx/snippets/ssl-params.conf;  
    ...  
}
```

5. Habilitar HTTP/2 (Passo 1, Parte II):

- Adicionar a primitiva `http2` à diretiva `listen` da porta 443 para habilitar o protocolo:

6. Nginx

None

```
listen 443 ssl http2;  
listen [::]:443 ssl http2;
```

7. **Redirecionamento:** É uma prática comum configurar um bloco de redirecionamento para forçar usuários que digitarem **HTTP** (`http://dominio`) a serem enviados para **HTTPS** (`https://dominio`).
8. **Outras Configurações Mencionadas:** Configurar páginas de erro personalizadas como **404** (Página não encontrada) e **503** (Página em manutenção), além de **autenticação**.